Project-Based Machine Learning & AI Roadmap

Professional 15-week implementation roadmap with 42 projects and task checklists

Completion rule: a project is complete only when all checklist items are done. Reaching a high score or successfully running the code is not, by itself, considered project completion.

Week 1 — NumPy / Matplotlib / Pandas

Project 1 — Store Sales Analysis

Dataset: Superstore Sales Dataset — Kaggle

Problem: Analyze sales, profit, customers and products to identify the strongest and weakest parts of the business.

To-Do

☐ Download the dataset and inspect the file structure.

☐ Load the data with Pandas.

☐ Inspect row/column counts.

☐ Inspect data types.

☐ Find missing values.

☐ Check and handle duplicates.

☐ Separate numerical and categorical columns.

☐ Calculate total sales.

☐ Calculate total profit.

☐ Calculate average order sales.

☐ Group sales by Category and Sub-Category.

☐ Find the 10 best-selling products.

☐ Find the 10 least-profitable products.

☐ Analyze sales by month.

☐ Plot Sales over Time.

☐ Plot sales by Category.

☐ Build a simple dashboard with several plots.

☐ Extract at least 5 real insights.

Project 2 — House Price EDA

Dataset: California Housing Dataset — sklearn

Problem: Understand which features relate to house prices and characterize the overall housing data.

To-Do

☐ Load the dataset.

☐ Inspect the DataFrame.

☐ Extract initial statistics.

☐ Plot the house-price distribution.

☐ Calculate mean, median and standard deviation.

☐ Calculate price quantiles.

☐ Inspect possible outliers.

☐ Plot boxplots for important features.

☐ Build a correlation matrix.

☐ Inspect cumulative distributions.

☐ Visualize each feature's relationship with the target.

☐ Identify the 5 features with the strongest correlation.

☐ Write a short EDA report.

Project 3 — NumPy Data Analysis Engine

Dataset: Sales data from Project 1

Problem: Repeat part of the previous analysis using NumPy rather than Pandas.

To-Do

☐ Convert the data to NumPy arrays.

☐ Practice indexing and slicing.

☐ Implement filtering with Boolean masks.

☐ Calculate the mean.

☐ Calculate the median.

☐ Calculate variance and standard deviation.

☐ Implement normalization.

☐ Implement Min-Max scaling.

☐ Practice matrix/vector operations.

☐ Build a function for statistical summaries.

☐ Compare your results with NumPy's built-in functions.

Week 2 — Statistics

Project 4 — Was the Marketing Campaign Effective?

Dataset: Marketing Campaign Dataset — Kaggle

Problem: Determine whether a marketing campaign actually increased customer purchases or whether the observed difference could be random.

To-Do

☐ Define the population and sample.

☐ Inspect sample size.

☐ Calculate mean and median before/after the campaign.

☐ Calculate variance and standard deviation.

☐ Calculate quantiles.

☐ Plot before/after boxplots.

☐ Define H0.

☐ Define H1.

☐ Choose a significance level.

☐ Choose the appropriate t-test.

☐ Run the test.

☐ Interpret the p-value.

☐ Report the statistical conclusion in plain language.

Project 5 — Income vs Education

Dataset: Adult Census Income Dataset — UCI/Kaggle

Problem: Investigate whether education level has a statistically significant relationship with income.

To-Do

☐ Clean the data.

☐ Define education groups.

☐ Compare income across groups.

☐ Visualize the distributions.

☐ Choose an appropriate correlation measure where applicable.

☐ Define statistical hypotheses.

☐ Select the appropriate statistical test.

☐ Run the test.

☐ Interpret the p-value.

☐ Support the conclusion with visualization.

Project 6 — Correlation & Distribution Lab

Dataset: Wine Quality Dataset — UCI

Problem: Build practical intuition for distributions, quantiles, outliers and correlation.

To-Do

☐ Identify important random variables.

☐ Inspect PMF for a suitable discrete variable.

☐ Plot histograms/PDF-style distributions for continuous variables.

☐ Plot CDFs.

☐ Calculate quantiles.

☐ Calculate IQR.

☐ Build boxplots.

☐ Calculate Pearson correlation.

☐ Calculate Spearman correlation.

☐ Compare the results.

☐ Report the strongest correlations.

Week 3 — Linear Algebra

Project 7 — Linear System Solver

Dataset: Synthetic Dataset

Problem: Solve a multivariate linear system using matrix operations.

To-Do

☐ Create vectors.

☐ Create matrices.

☐ Implement vector addition.

☐ Implement dot product.

☐ Implement matrix multiplication.

☐ Calculate norms.

☐ Inspect transposes.

☐ Create a system Ax=b.

☐ Solve it with NumPy.

☐ Verify the solution.

Project 8 — PCA from Scratch

Dataset: Wine Dataset

Problem: Reduce dimensionality while preserving as much information as possible.

To-Do

☐ Standardize the data.

☐ Build the covariance matrix.

☐ Calculate eigenvalues.

☐ Calculate eigenvectors.

☐ Sort eigenvalues.

☐ Select principal components.

☐ Project the data into the new space.

☐ Calculate explained variance.

☐ Visualize the data in 2D.

☐ Compare with sklearn.PCA.

Project 9 — Image Compression with SVD

Dataset: A grayscale image from skimage.data

Problem: Use SVD to reconstruct an image at different ranks and study the compression/quality trade-off.

To-Do

☐ Convert the image to grayscale.

☐ Extract the image matrix.

☐ Compute SVD.

☐ Inspect singular values.

☐ Reconstruct using different ranks.

☐ Test rank 5.

☐ Test rank 20.

☐ Test rank 50.

☐ Compare reconstruction quality.

☐ Calculate compression ratio.

☐ Visualize the results.

Week 4 — Preprocessing / Classification

Project 10 — Iris Classification

Dataset: Iris Dataset — sklearn

Problem: Build and compare basic classification models.

To-Do

☐ Load the dataset.

☐ Perform EDA.

☐ Inspect features.

☐ Inspect class distribution.

☐ Perform train/test split.

☐ Apply scaling.

☐ Build Logistic Regression.

☐ Build KNN.

☐ Build Decision Tree.

☐ Compare accuracy.

☐ Build a confusion matrix.

☐ Select the best model.

Project 11 — Handwritten Digit Classification

Dataset: Digits Dataset — sklearn

Problem: Classify handwritten digits and analyze model errors.

To-Do

☐ Visualize images.

☐ Inspect pixel features.

☐ Split train/test data.

☐ Check the effect of scaling.

☐ Run Logistic Regression.

☐ Run KNN.

☐ Run Decision Tree.

☐ Run Random Forest.

☐ Calculate accuracy, precision, recall and F1.

☐ Build a confusion matrix.

☐ Display misclassified examples.

Project 12 — Customer Churn Prediction

Dataset: Telco Customer Churn — IBM/Kaggle

Problem: Identify telecom customers who are likely to cancel their service.

To-Do

☐ Inspect missing values.

☐ Define the target.

☐ Separate numerical/categorical features.

☐ Encode categorical variables.

☐ Scale numerical variables.

☐ Split train/test data.

☐ Build Logistic Regression.

☐ Build KNN.

☐ Build Decision Tree.

☐ Build a confusion matrix.

☐ Calculate precision, recall and F1.

☐ Analyze the business impact of false negatives.

☐ Select the appropriate business model.

Week 5 — SVM / Cross Validation / Grid Search

Project 13 — Breast Cancer Classification

Dataset: Breast Cancer Wisconsin Dataset — sklearn

Problem: Compare SVM kernels and understand the effect of C and gamma.

To-Do

☐ Inspect the dataset.

☐ Split train/test.

☐ Scale features.

☐ Build Linear SVM.

☐ Build RBF SVM.

☐ Build Polynomial SVM.

☐ Vary C.

☐ Vary gamma.

☐ Inspect a 2D decision boundary where appropriate.

☐ Compare accuracy, precision, recall and F1.

☐ Select the best kernel.

Project 14 — Model Selection with Cross Validation

Dataset: Breast Cancer Dataset

Problem: Evaluate several classifiers using the same cross-validation protocol.

To-Do

☐ Run 5-fold cross-validation.

☐ Evaluate Logistic Regression.

☐ Evaluate KNN.

☐ Evaluate SVM.

☐ Evaluate Decision Tree.

☐ Evaluate Random Forest.

☐ Calculate mean CV score.

☐ Inspect standard deviation.

☐ Use the test set only at the end.

Project 15 — Hyperparameter Optimization

Dataset: Same dataset as Project 14

Problem: Use systematic hyperparameter search and determine whether tuning materially improves performance.

To-Do

☐ Identify model hyperparameters.

☐ Define search spaces.

☐ Run GridSearchCV.

☐ Extract the best parameters.

☐ Record the best CV score.

☐ Evaluate the final model on the test set.

☐ Compare before/after tuning.

☐ Assess whether tuning was worthwhile.

Week 6 — Regression

Project 16 — House Price Prediction

Dataset: California Housing — sklearn

Problem: Build a regression model that predicts house prices.

To-Do

☐ Perform EDA.

☐ Define the target.

☐ Split train/test.

☐ Train Linear Regression.

☐ Calculate MAE.

☐ Calculate MSE.

☐ Calculate RMSE.

☐ Calculate R^2.

☐ Plot prediction vs actual.

☐ Plot residuals.

☐ Inspect important features.

Project 17 — Polynomial Regression

Dataset: California Housing — selected features

Problem: Observe underfitting and overfitting by increasing polynomial model complexity.

To-Do

☐ Select a primary feature.

☐ Train Linear Regression.

☐ Train degree-2 polynomial regression.

☐ Train degree-3.

☐ Train degree-5.

☐ Train degree-10.

☐ Plot train/validation error.

☐ Identify underfitting.

☐ Identify overfitting.

☐ Select an appropriate degree.

Project 18 — Regression Model Comparison

Dataset: California Housing

Problem: Compare classical regression models under a consistent evaluation protocol.

To-Do

☐ Train Linear Regression.

☐ Train Polynomial Regression.

☐ Train Decision Tree Regressor.

☐ Train Random Forest Regressor.

☐ Use cross-validation.

☐ Compare MAE.

☐ Compare RMSE.

☐ Compare R^2.

☐ Perform prediction error analysis.

☐ Select the final model.

Week 7 — Clustering

Project 19 — Customer Segmentation

Dataset: Mall Customers — Kaggle

Problem: Segment customers into meaningful behavioral groups.

To-Do

☐ Perform EDA.

☐ Select features.

☐ Scale features.

☐ Run K-Means with multiple K values.

☐ Calculate inertia.

☐ Use the elbow method.

☐ Calculate silhouette score.

☐ Select K.

☐ Visualize clusters.

☐ Build a customer profile for each cluster.

Project 20 — K-Means vs DBSCAN

Dataset: Mall Customers + Synthetic 2D Dataset

Problem: Compare centroid-based and density-based clustering.

To-Do

☐ Run K-Means.

☐ Run DBSCAN.

☐ Vary eps.

☐ Vary min_samples.

☐ Identify noise points.

☐ Compare cluster counts.

☐ Compare silhouette scores.

☐ Visualize both approaches.

☐ Report strengths and limitations.

Project 21 — Outlier Detection

Dataset: Credit Card Fraud Dataset — Kaggle

Problem: Compare multiple unsupervised methods for identifying anomalous observations.

To-Do

☐ Inspect distributions.

☐ Implement Z-score.

☐ Implement IQR.

☐ Try DBSCAN.

☐ Try Local Outlier Factor.

☐ Try One-Class SVM.

☐ Compare detected outliers.

☐ Inspect false positives where labels permit.

☐ Select the most suitable approach.

Week 8 — Pipelines / Gradient Descent

Project 22 — Production-Style ML Pipeline

Dataset: Adult Census Income

Problem: Build a reusable preprocessing + modeling pipeline.

To-Do

☐ Identify numerical/categorical features.

☐ Define missing-value strategies.

☐ Build a numerical imputer.

☐ Build a categorical imputer.

☐ Apply one-hot encoding.

☐ Apply StandardScaler.

☐ Build a ColumnTransformer.

☐ Build a Pipeline.

☐ Add a classifier.

☐ Run cross-validation.

☐ Run grid search.

☐ Save the complete pipeline.

☐ Test it on new data.

Project 23 — Linear Regression with Gradient Descent

Dataset: California Housing — one or more features

Problem: Implement the core optimization loop yourself.

To-Do

☐ Define the mathematical model.

☐ Define the loss function.

☐ Initialize weights.

☐ Define the learning rate.

☐ Write the prediction function.

☐ Calculate MSE.

☐ Calculate gradients.

☐ Implement weight updates.

☐ Build the training loop.

☐ Plot the loss curve.

☐ Test different learning rates.

☐ Compare with LinearRegression.

Project 24 — Gradient Descent Experiment Lab

Dataset: Synthetic Regression Dataset

Problem: Study convergence behavior under different optimization settings.

To-Do

☐ Generate a synthetic dataset.

☐ Define a linear model.

☐ Test a small learning rate.

☐ Test an appropriate learning rate.

☐ Test a large learning rate.

☐ Plot the loss curves.

☐ Inspect convergence.

☐ Change the number of iterations.

☐ Implement batch gradient descent.

☐ Implement mini-batch gradient descent.

Week 9 — Neural Networks

Project 25 — Neural Network from Scratch

Dataset: XOR + MNIST subset

Problem: Implement a small feed-forward neural network without a deep-learning framework.

To-Do

☐ Build a Dense layer.

☐ Initialize weights.

☐ Add bias.

☐ Implement forward propagation.

☐ Implement an activation function.

☐ Define a loss function.

☐ Calculate gradients.

☐ Implement backpropagation.

☐ Implement weight updates.

☐ Build the training loop.

☐ Solve XOR.

☐ Try the network on a small MNIST subset.

Project 26 — Digit Classification with Keras

Dataset: MNIST

Problem: Build, train and diagnose a dense neural network.

To-Do

☐ Load MNIST.

☐ Normalize inputs.

☐ Create train/validation/test splits.

☐ Build a dense network.

☐ Choose activations.

☐ Choose an optimizer.

☐ Define the loss.

☐ Train the model.

☐ Inspect training history.

☐ Evaluate validation performance.

☐ Build a confusion matrix.

☐ Display misclassified images.

Project 27 — Fashion-MNIST Experiment Lab

Dataset: Fashion-MNIST

Problem: Study architecture and regularization choices experimentally.

To-Do

☐ Build a baseline network.

☐ Change the number of layers.

☐ Change neuron counts.

☐ Compare optimizers.

☐ Change learning rate.

☐ Change batch size.

☐ Add dropout.

☐ Test L1 regularization.

☐ Test L2 regularization.

☐ Use early stopping.

☐ Analyze validation curves.

☐ Select the best architecture.

Week 10 — CNN / Data Augmentation

Project 28 — CIFAR-10 Image Classification

Dataset: CIFAR-10

Problem: Build a CNN and perform systematic image-classification error analysis.

To-Do

☐ Load CIFAR-10.

☐ Visualize images.

☐ Normalize inputs.

☐ Build a CNN baseline.

☐ Use Conv2D.

☐ Add activations.

☐ Use pooling.

☐ Add multiple convolutional blocks.

☐ Build a dense classifier.

☐ Train.

☐ Validate.

☐ Build a confusion matrix.

☐ Analyze class-wise performance.

☐ Analyze errors.

Project 29 — CNN Architecture Lab

Dataset: CIFAR-10

Problem: Study how CNN architecture choices affect generalization and training.

To-Do

☐ Build a simple CNN.

☐ Build a deeper CNN.

☐ Change kernel size.

☐ Change filter counts.

☐ Change pooling.

☐ Add dropout.

☐ Test batch normalization.

☐ Compare models under a fixed protocol.

☐ Plot accuracy/loss curves.

☐ Select the best architecture.

Project 30 — Real-World Image Classification

Dataset: Intel Image Classification — Kaggle

Problem: Classify environmental scenes using a complete image pipeline.

To-Do

☐ Inspect the dataset.

☐ Inspect class distribution.

☐ Inspect image resolution.

☐ Visualize samples.

☐ Create train/validation splits.

☐ Preprocess images.

☐ Build a baseline CNN.

☐ Add data augmentation.

☐ Train the augmented model.

☐ Inspect overfitting.

☐ Build a confusion matrix.

☐ Perform error analysis.

Week 11 — Transfer Learning / Functional API

Project 31 — Transfer Learning

Dataset: Intel Image Classification

Problem: Use a pretrained vision backbone and compare it with the scratch CNN.

To-Do

☐ Choose a pretrained model.

☐ Load the backbone.

☐ Load pretrained weights.

☐ Freeze the backbone.

☐ Build a classification head.

☐ Train the model.

☐ Compare with Project 30.

☐ Compare training time.

☐ Inspect parameter counts.

Project 32 — Fine-Tuning

Dataset: Intel Image Classification

Problem: Fine-tune part of a pretrained network to improve task-specific performance.

To-Do

☐ Select the best transfer-learning model.

☐ Unfreeze selected upper layers.

☐ Reduce the learning rate.

☐ Fine-tune.

☐ Monitor validation performance.

☐ Compare scratch CNN vs transfer learning vs fine-tuning.

☐ Perform error analysis.

Project 33 — Multi-Input House Price Model

Dataset: California Housing + a categorical supplementary feature

Problem: Build a Keras Functional API model that merges numerical and categorical inputs.

To-Do

☐ Define numerical input.

☐ Define categorical input.

☐ Create a branch for each input.

☐ Build dense layers.

☐ Merge the branches.

☐ Build a regression output.

☐ Use the Functional API.

☐ Inspect model summary.

☐ Train.

☐ Evaluate.

☐ Compare with a Sequential model.

Week 12 — NLP / Embeddings

Project 34 — IMDB Sentiment Analysis

Dataset: IMDB Movie Reviews — Keras

Problem: Build increasingly expressive text classifiers.

To-Do

☐ Load the dataset.

☐ Inspect review text.

☐ Inspect vocabulary.

☐ Tokenize.

☐ Pad sequences.

☐ Build a bag-of-words baseline.

☐ Build a dense classifier.

☐ Add an embedding layer.

☐ Train Embedding + Dense.

☐ Calculate accuracy/F1.

☐ Perform error analysis.

☐ Inspect misclassified examples.

Project 35 — Persian News Classification

Dataset: Persian News Dataset — Hamshahri / Persian News Classification Dataset

Problem: Classify Persian news into topics and compare classical NLP with neural representations.

To-Do

☐ Prepare the dataset.

☐ Inspect the texts.

☐ Normalize Persian text.

☐ Remove unnecessary characters.

☐ Tokenize with Hazm.

☐ Handle stopwords.

☐ Split train/test.

☐ Build a TF-IDF baseline.

☐ Train Logistic Regression.

☐ Explore word embeddings.

☐ Build Embedding + Neural Network.

☐ Calculate accuracy/F1.

☐ Build a confusion matrix.

☐ Analyze model errors.

Project 36 — IMDB with Embedding + CNN

Dataset: IMDB

Problem: Compare a simple embedding model with a 1D convolutional text classifier.

To-Do

☐ Tokenize.

☐ Pad sequences.

☐ Add an embedding layer.

☐ Add Conv1D.

☐ Add pooling.

☐ Add a dense output.

☐ Train.

☐ Validate.

☐ Compare with Embedding + Dense.

☐ Test kernel sizes.

☐ Perform error analysis.

Week 13 — RNN / LSTM / GRU

Project 37 — Time-Series Forecasting

Dataset: Jena Climate Dataset

Problem: Predict a future climate variable from historical sequences.

To-Do

☐ Inspect timestamps.

☐ Inspect temporal features.

☐ Handle missing values.

☐ Normalize data.

☐ Create sliding windows.

☐ Choose sequence length.

☐ Split data chronologically.

☐ Build a SimpleRNN.

☐ Generate predictions.

☐ Calculate MAE/RMSE.

☐ Plot actual vs predicted values.

Project 38 — RNN vs LSTM vs GRU

Dataset: IMDB

Problem: Compare recurrent architectures under a controlled experimental setup.

To-Do

☐ Add an embedding layer.

☐ Build SimpleRNN.

☐ Build LSTM.

☐ Build GRU.

☐ Train all models under comparable conditions.

☐ Compare accuracy.

☐ Compare training time.

☐ Change sequence length.

☐ Test recurrent dropout.

☐ Inspect overfitting.

☐ Select the best architecture.

Week 14 — Autoencoder / U-Net / Segmentation

Project 39 — Dense Autoencoder

Dataset: MNIST

Problem: Learn a compact latent representation and reconstruct handwritten digits.

To-Do

☐ Prepare MNIST.

☐ Flatten inputs.

☐ Build an encoder.

☐ Create a bottleneck.

☐ Build a decoder.

☐ Define reconstruction loss.

☐ Train.

☐ Compare original vs reconstructed images.

☐ Change latent dimension.

☐ Analyze the effect of latent size.

Project 40 — Denoising Convolutional Autoencoder

Dataset: MNIST

Problem: Recover clean images from noisy inputs.

To-Do

☐ Generate noisy images.

☐ Build a convolutional encoder.

☐ Create a bottleneck.

☐ Build a convolutional decoder.

☐ Train noisy-image → clean-image reconstruction.

☐ Visualize reconstructions.

☐ Test different noise levels.

☐ Calculate MSE.

☐ Compare with the dense autoencoder.

Project 41 — U-Net Image Segmentation

Dataset: Oxford-IIIT Pet Dataset

Problem: Segment the foreground animal in each image.

To-Do

☐ Prepare the dataset.

☐ Inspect image/mask pairs.

☐ Resize images and masks.

☐ Normalize images.

☐ Create train/validation splits.

☐ Build an encoder.

☐ Build a bottleneck.

☐ Build a decoder.

☐ Add skip connections.

☐ Implement U-Net with the Functional API.

☐ Define segmentation loss.

☐ Calculate Dice score.

☐ Calculate IoU.

☐ Visualize predicted masks.

☐ Analyze failure cases.

Week 15 — Deployment / ONNX / Recommender Systems

Project 42A — ML Model → API

Dataset: Best model from an earlier project

Problem: Turn a trained model into a usable prediction service.

To-Do

☐ Select the final model.

☐ Save the model.

☐ Save preprocessing with the model.

☐ Build a loading pipeline.

☐ Write a predict() function.

☐ Design JSON input.

☐ Set up FastAPI.

☐ Create a /predict endpoint.

☐ Add request validation.

☐ Design a clear response.

☐ Test with Postman or curl.

☐ Add error handling.

Project 42B — ONNX Inference

Dataset: Same deployment model

Problem: Export a trained model and run inference with ONNX Runtime.

To-Do

☐ Export the model to ONNX.

☐ Load the ONNX model.

☐ Run inference with ONNX Runtime.

☐ Compare predictions with the original model.

☐ Compare inference time.

☐ Inspect numerical differences.

Project 42C — Content-Based Movie Recommender

Dataset: MovieLens Dataset

Problem: Recommend similar movies based on movie content.

To-Do

☐ Load movie metadata.

☐ Prepare genres/features.

☐ Build feature representations.

☐ Create movie vectors.

☐ Calculate cosine similarity.

☐ Build a similarity matrix.

☐ Write recommend(movie).

☐ Generate top-5 recommendations.

☐ Generate top-10 recommendations.

☐ Manually inspect several recommendations.

Project 42D — Collaborative Filtering

Dataset: MovieLens Dataset

Problem: Recommend movies based on user behavior rather than movie content alone.

To-Do

☐ Build a user-movie matrix.

☐ Inspect sparsity.

☐ Calculate user similarity.

☐ Calculate item similarity.

☐ Build rating predictions.

☐ Generate top-N recommendations.

☐ Compare with content-based recommendations.

☐ Investigate the cold-start problem.

☐ Document system limitations.

Capstone Project — Weeks 16–17

Build one complete end-to-end intelligent system. Choose one path: Computer Vision, NLP, or Tabular ML.

Phase 1 — Problem Definition

☐ Define the business/research problem.

☐ Specify inputs.

☐ Specify the target.

☐ Choose evaluation metrics.

☐ Define a baseline.

Phase 2 — Data

☐ Acquire the dataset.

☐ Perform data inspection.

☐ Perform EDA.

☐ Handle missing values.

☐ Inspect outliers.

☐ Inspect class imbalance.

☐ Create train/validation/test splits.

Phase 3 — Baseline

☐ Build the simplest reasonable model.

☐ Record baseline metrics.

☐ Perform initial error analysis.

Phase 4 — Advanced Model

☐ Design a stronger model.

☐ Perform hyperparameter tuning.

☐ Use cross-validation when appropriate.

☐ Apply regularization.

☐ Apply data augmentation when appropriate.

☐ Use early stopping.

Phase 5 — Evaluation

☐ Report final metrics.

☐ Build a confusion matrix when applicable.

☐ Perform error analysis.

☐ Inspect worst predictions.

☐ Inspect best predictions.

☐ Document failure cases.

☐ Compare against the baseline.

Phase 6 — Model Engineering

☐ Build the preprocessing pipeline.

☐ Save the model.

☐ Load the model.

☐ Create an inference function.

☐ Test inference.

Phase 7 — Deployment

☐ Build a FastAPI service.

☐ Create /predict.

☐ Validate inputs.

☐ Handle errors.

☐ Test the API.

☐ Export to ONNX when appropriate.

☐ Test ONNX Runtime inference.

Phase 8 — Final Repository

☐ Create README.md.

☐ Create requirements.txt.

☐ Create .gitignore.

☐ Organize data/.

☐ Organize notebooks/.

☐ Organize src/.

☐ Organize models/.

☐ Organize app/.

☐ Organize tests/.

☐ Document architecture.

☐ Document the dataset.

☐ Document training.

☐ Document results.

☐ Document error analysis.

☐ Document API examples.

☐ Document limitations.

☐ Document future work.

Recommended repository structure

Project/

├── README.md

├── requirements.txt

├── .gitignore

├── data/

├── notebooks/

│ ├── 01_EDA.ipynb

│ ├── 02_Preprocessing.ipynb

```
│   ├── 03_Baseline.ipynb
│   ├── 04_Model.ipynb
│   └── 05_Evaluation.ipynb
├── src/
│   ├── preprocessing.py
│   ├── model.py
│   ├── train.py
│   └── evaluate.py
├── models/
├── app/
│   └── main.py
└── tests/
    └── test_model.py
```

Learning progression: Python → NumPy/Pandas → Statistics → Linear Algebra → Preprocessing → Classification → Regression → Clustering → PCA → Gradient Descent → Neural Networks → CNN → Transfer Learning → Functional API → NLP/Embeddings → RNN/LSTM/GRU → Autoencoders → U-Net/Segmentation → Deployment/ONNX → Recommender Systems → End-to-End Capstone.